



The *xraylib* library for X-ray–matter interaction cross sections

Tom Schoonjans

Monte Carlo simulation tools for X-ray imaging and fluorescence Workshop
24 & 25 February 2014

Outline

1. Introduction and history
2. Current development status
 1. Available datasets
 2. Platforms
 3. Language bindings
3. Typical usage examples
4. Command-line interface
5. Web-interface
6. Future developments

A brief introduction to *xraylib*

The quest for an X-ray parameter library...

1. Ease of use

Preferably software that can be linked with our own software

2. Reliable data

Most compilations are based on a selection of values considered to be reliable by the authors

XCOM

- NIST: Berger and Hubbell
- Cross sections for scattering and photo ionization
- Compiles into an executable, not a library
- Fortran 77

mucal

- Illinois Institute of Technology: written by Pathikrit Bandyopadhyay
- McMaster and Krause
- Edge energies, fluorescence yields, scattering and photoionization cross sections
- Exports only one function for all types of data
- Fortran 77 and C

xraylib

ESRF/University of Sassari

A. Brunetti, M. Sanchez del Rio, B. Golosio, A. Simionovici and
A. Somogyi, Spectrochim. Acta Part B., 59, 1725-1731 (2004).

Origins of *xraylib*: 2002-2005

- Created by Bruno Golosio to support his work on Monte-Carlo simulations and XRF tomography reconstructions
- Contributions from Manuel Sanchez del Rio, Alexandre Simionovici
- Databases taken from Krause, Cullen, Larskin, Deslattes, Hubbell
- Bindings for IDL and Python

xraylib: current development status

xraylib: general characteristics

- Core library written in ANSI-C
- Bindings for many languages
- Very simple and light interface
- Extensive use of macros (KL3_LINE, K_SHELL, K3_L2M5_AUGER etc...)
- Thread-safe!
- BSD license: very liberal...
- Hosted on www.github.com/tschoonj/xraylib
- Comprehensive documentation

List of available datasets: part I

- Atomic weight
- (Partial) Photoionization cross sections
- Rayleigh cross sections
- Compton cross sections
- XRF cross sections
- Klein-Nishina CS
- Differential cross sections: polarized and unpolarized
- Absorption edges
- XRF line energies
- Jump factors
- Radiative rates
- Coster-Kronig transition probabilities

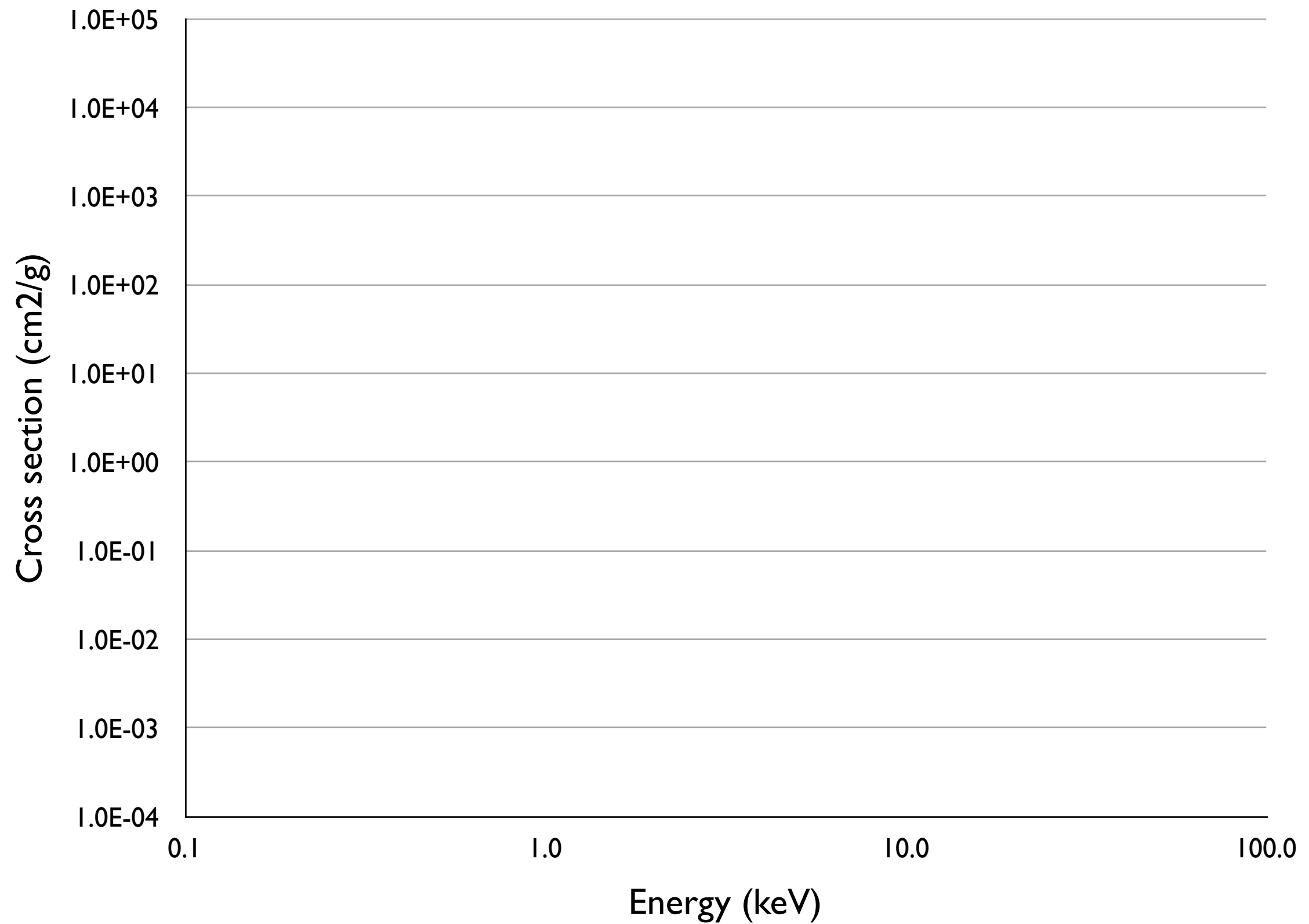
List of available datasets: part 2

- Auger non-radiative rates
- Atomic level widths
- Electronic configurations
- Anomalous scattering
- Atomic form factors
- Incoherent scattering function
- Compton broadening profiles
- Refractive indices
- Crystal diffraction: structure factors, Bragg angles, unit cell volumes, d-spacing, Q scattering amplitude
- Elemental density
- Mass-energy absorption cross sections

Utility functions

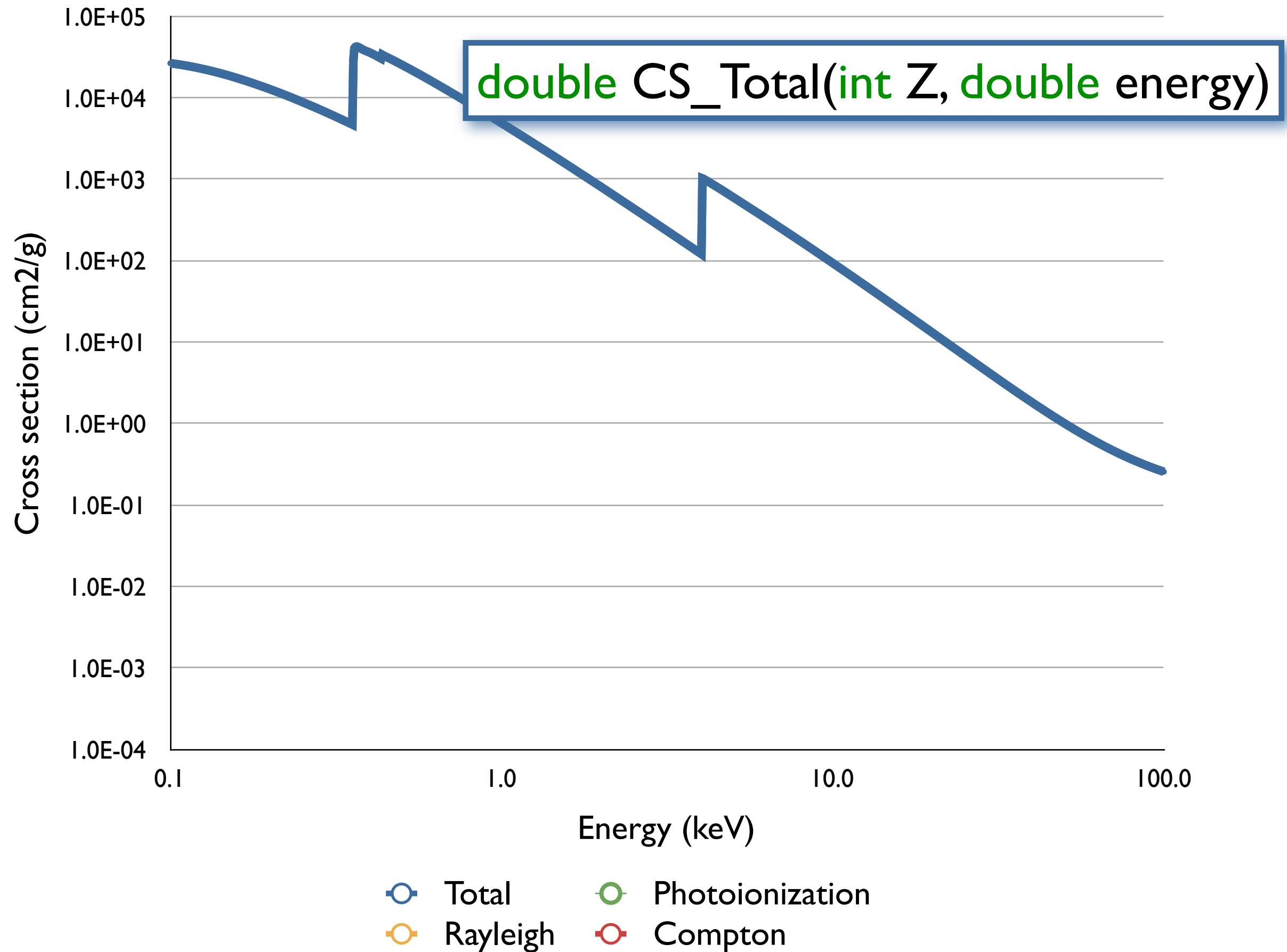
- Compound parser for chemical formulas: support for brackets and real indices
- Builtin composition and densities of the NIST compound list
- Many functions support passing chemical formulas and NIST compound names!

Ca

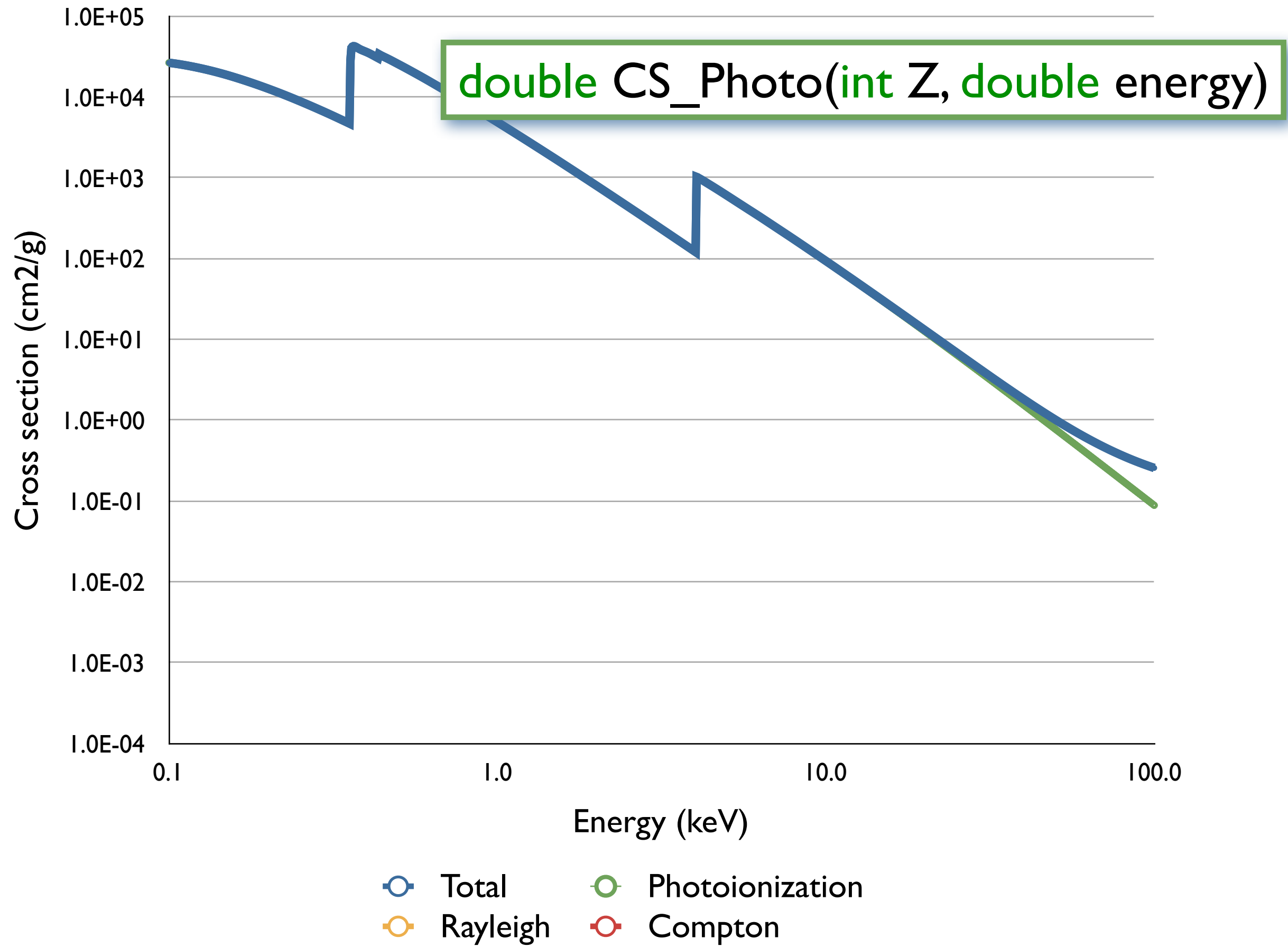


- Total
- Rayleigh
- Photoionization
- Compton

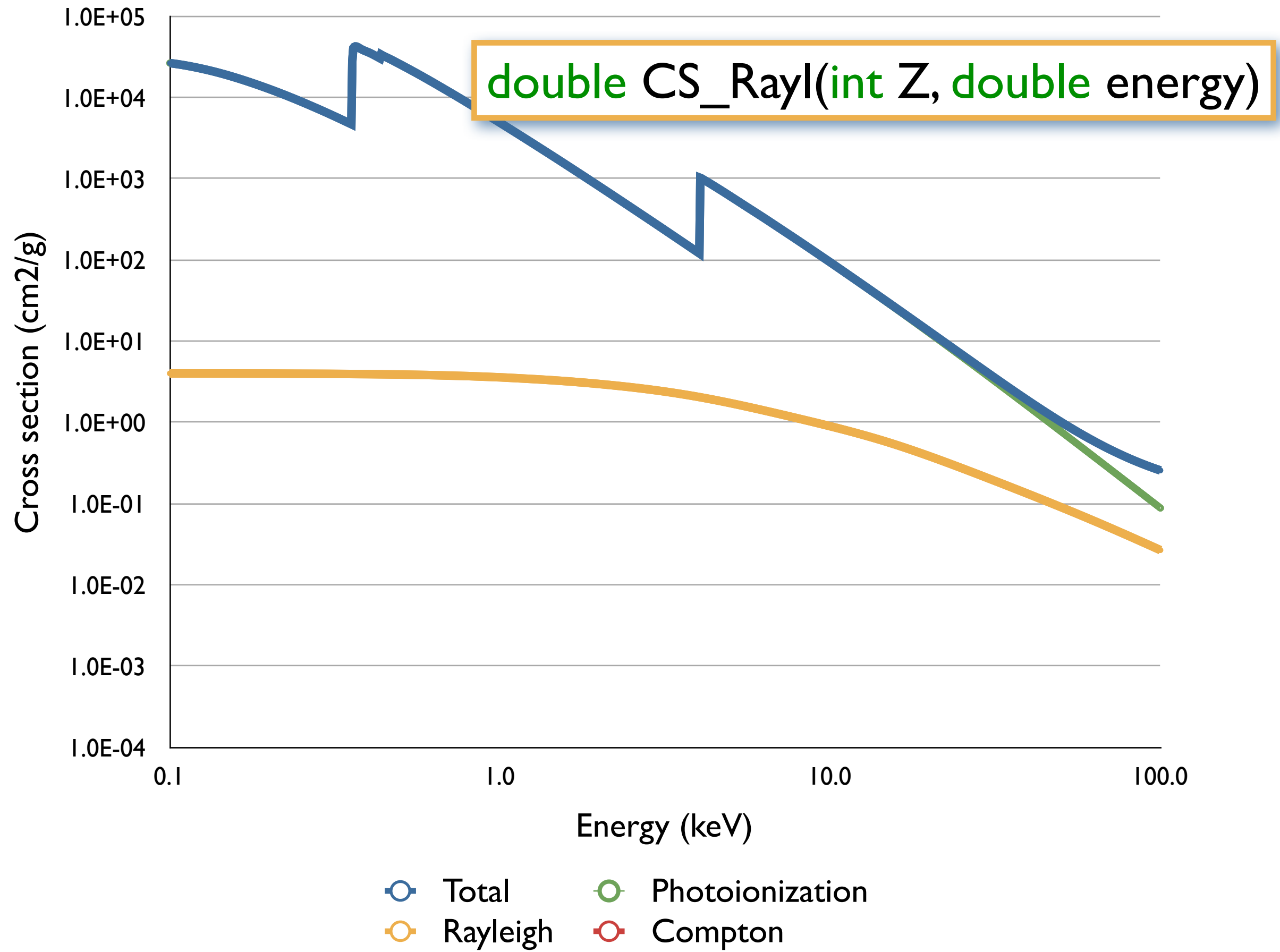
Ca



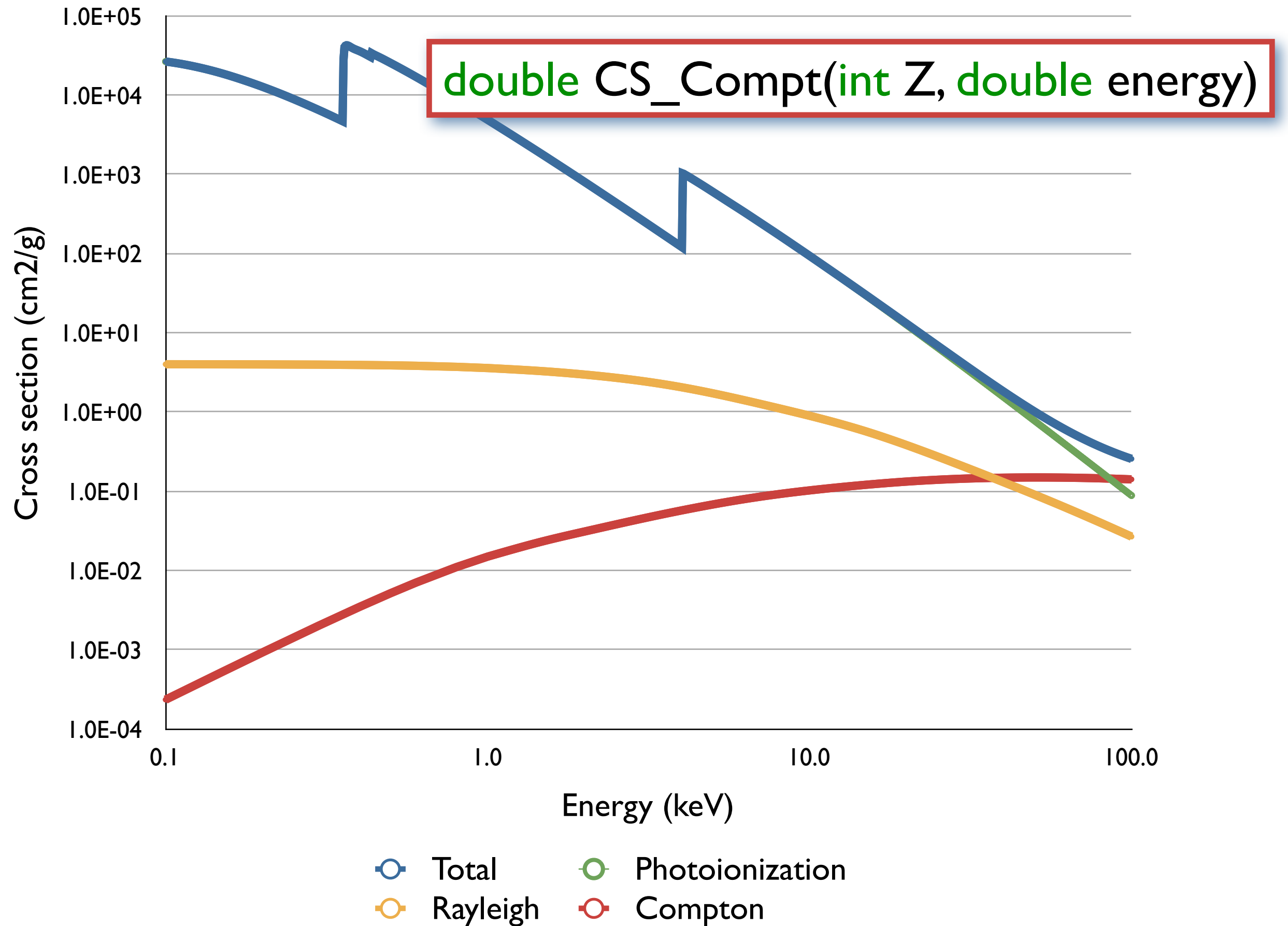
Ca



Ca

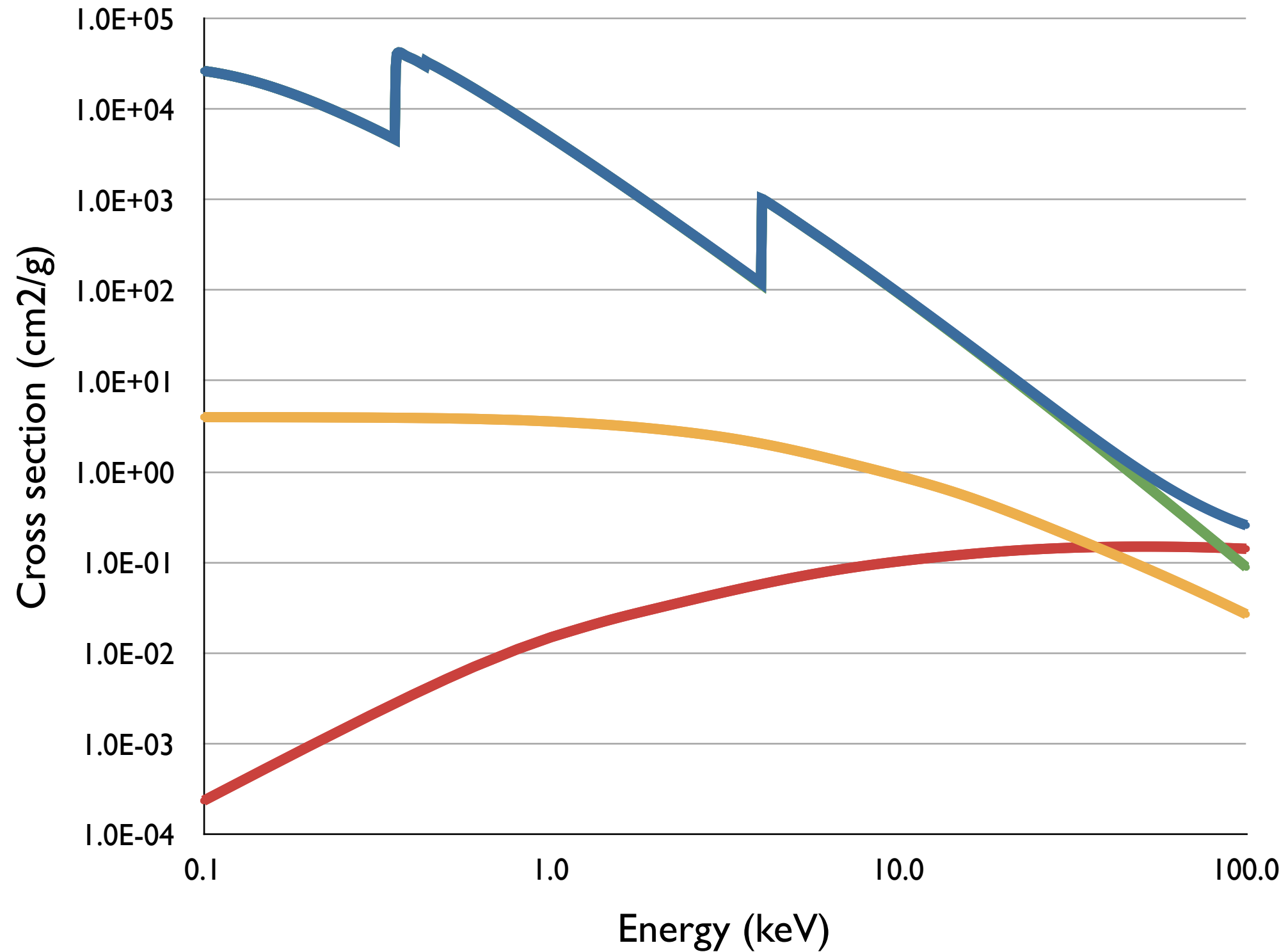


Ca



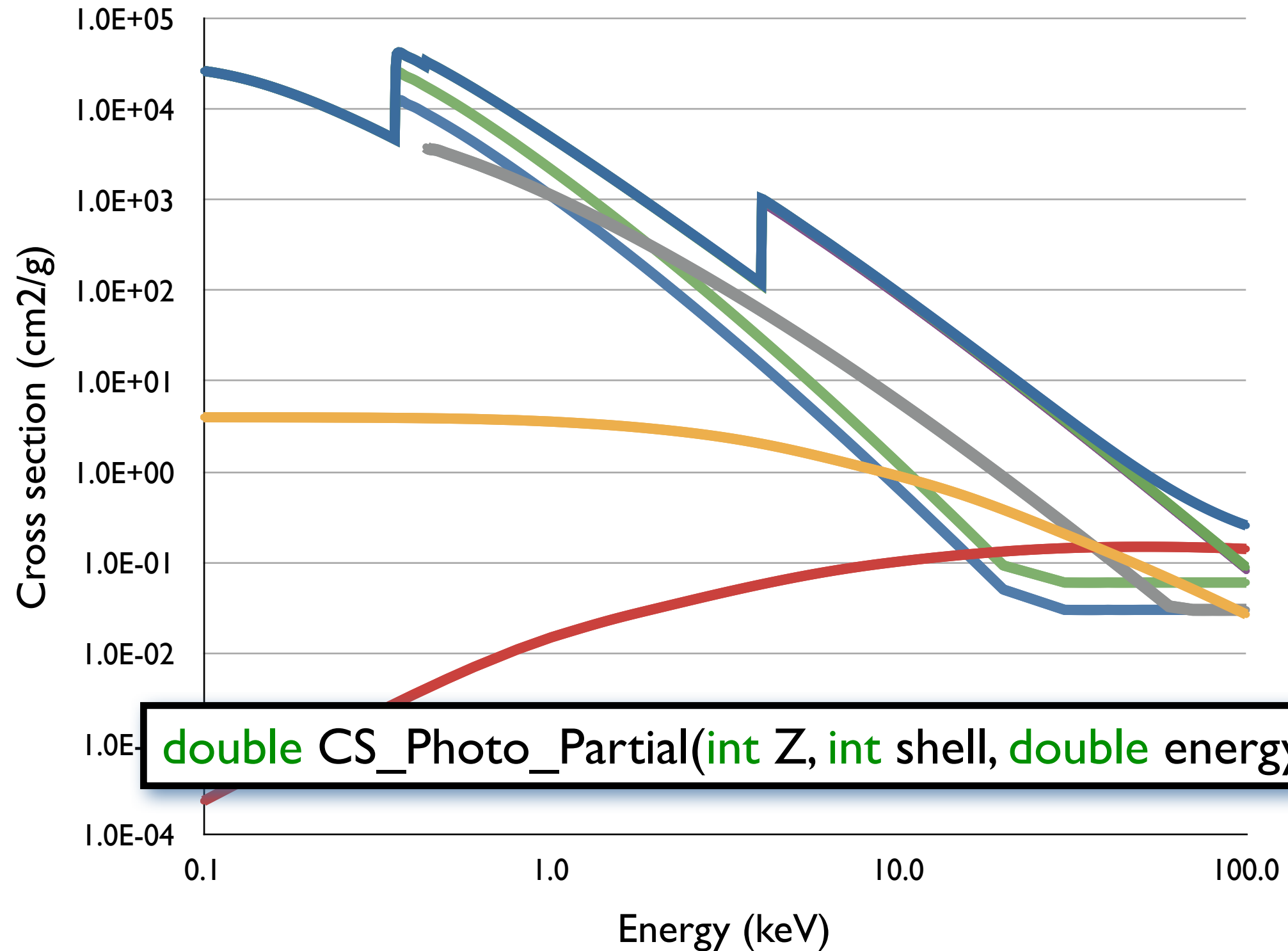
- | | | |
|--|--|---|
|  Total |  Photoionization |  Rayleigh |
|  Compton |  Photoionization-K |  Photoionization-L I |
|  Photoionization-L2 |  Photoionization-L3 | |

Ca



- Total
- Compton
- Photoionization-L2
- Photoionization
- Photoionization-K
- Photoionization-L3
- Rayleigh
- Photoionization-L1

Ca



- | | | |
|--------------------|--------------------|--------------------|
| Total | Photoionization | Rayleigh |
| Compton | Photoionization-K | Photoionization-L1 |
| Photoionization-L2 | Photoionization-L3 | |

XRF cross sections: initial implementation

- `double CS_FluorLine(int Z, int line, double energy)`
- Partial photoelectric CS using jump ratios
- Fluorescence yields
- Radiative rates

$$Q_{K\alpha} = \sigma_K \omega_K p_{K\alpha}$$

$$Q_{L_{1j}} = \sigma_{L_1} \omega_{L_1} p_{L_{1j}}$$

$$Q_{L_{2j}} = \sigma_{L_2} \omega_{L_2} p_{L_{2j}} + \sigma_{L_1} \omega_{L_2} p_{L_{2j}} f_{1,2}$$

$$Q_{L_{3j}} = \sigma_{L_3} \omega_{L_3} p_{L_{3j}} + \sigma_{L_2} \omega_{L_3} p_{L_{3j}} f_{2,3} + \sigma_{L_1} \omega_{L_3} p_{L_{3j}} (f_{1,3} + f_{1,2} f_{2,3})$$

XRF cross sections: new implementation

- `double CS_FluorLine_Kissel(int Z, int line, double energy)`
- Partial photoelectric CS using theoretically predicted values
- Fluorescence yields
- Radiative rates

$$Q_{K\alpha} = \sigma_K \omega_K p_{K\alpha}$$

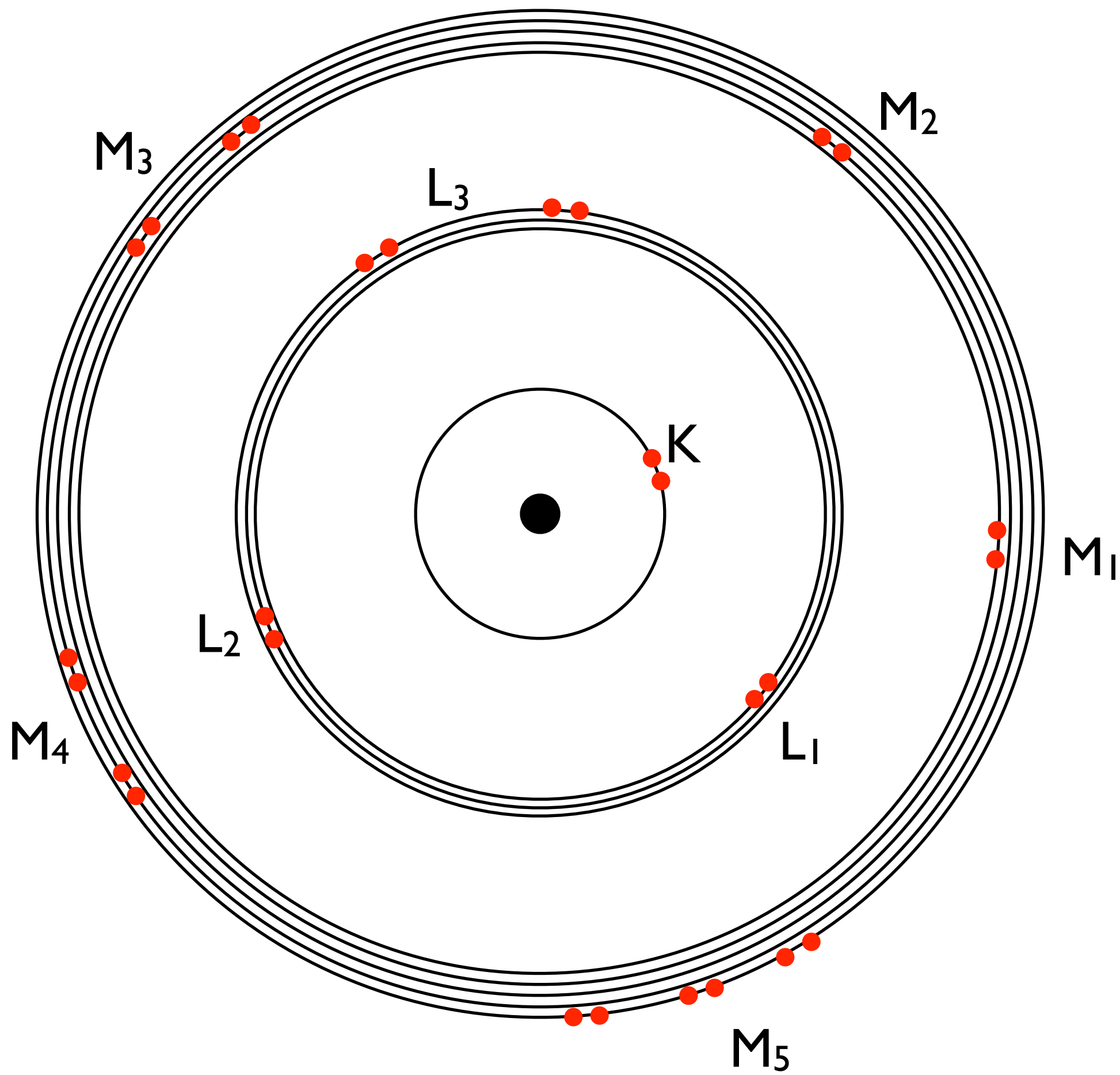
$$Q_{L_{1j}} = \sigma_{L_1} \omega_{L_1} p_{L_{1j}}$$

$$Q_{L_{2j}} = \sigma_{L_2} \omega_{L_2} p_{L_{2j}} + \sigma_{L_1} \omega_{L_2} p_{L_{2j}} f_{1,2}$$

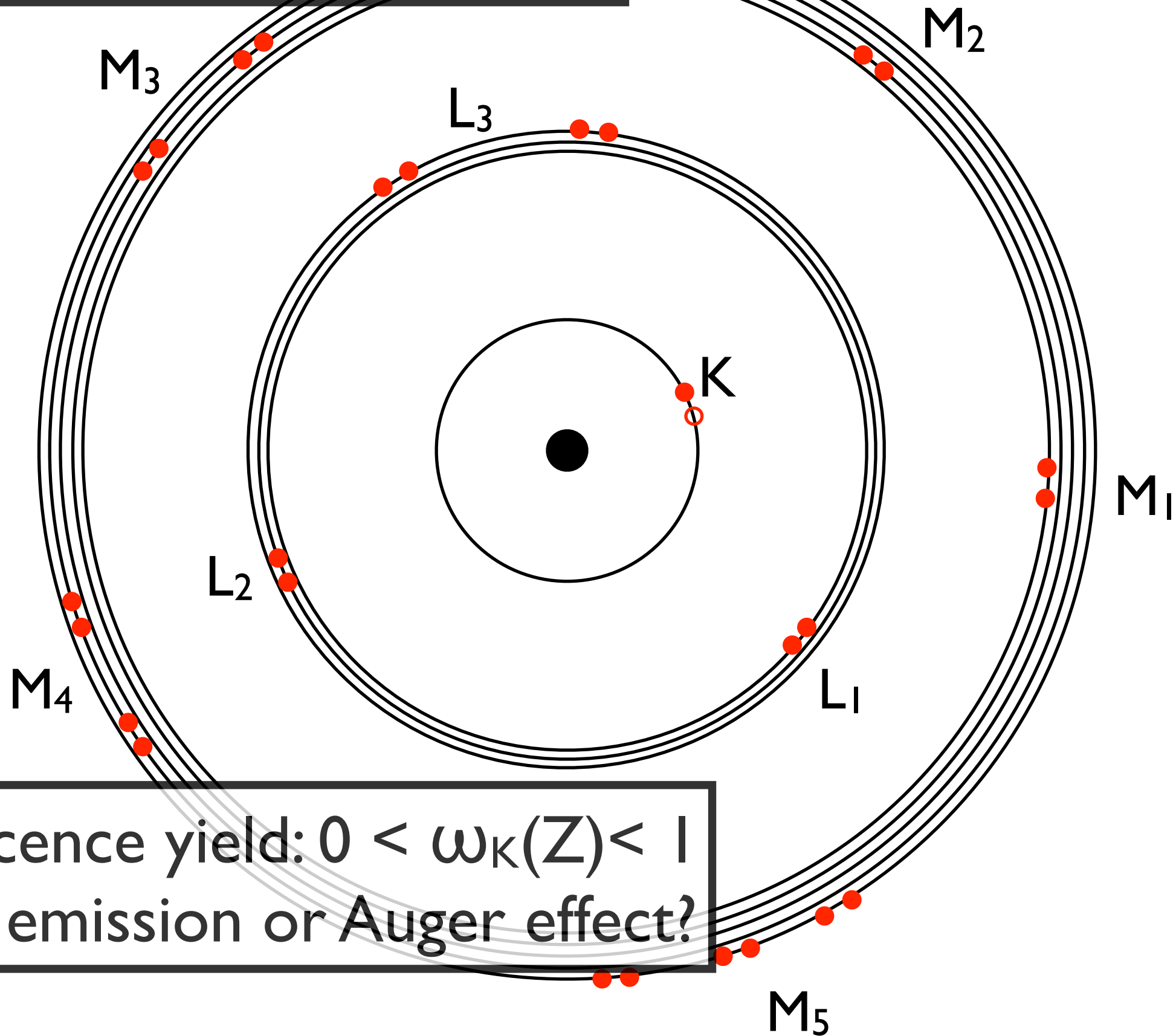
$$Q_{L_{3j}} = \sigma_{L_3} \omega_{L_3} p_{L_{3j}} + \sigma_{L_2} \omega_{L_3} p_{L_{3j}} f_{2,3} + \sigma_{L_1} \omega_{L_3} p_{L_{3j}} (f_{1,3} + f_{1,2} f_{2,3})$$

XRF cross sections: cascade effect

- Occurs whenever multiple shells of a particular element can be excited
- Two components: radiative and non-radiative
- Leads to considerable boost in the observed intensity of L- and M-lines: several times larger than intensity through primary excitations
- Very obvious when using monochromatic excitations
- Very often neglected in quantification and simulations
- Complex implementation
- Many fundamental parameters involved → accuracy?

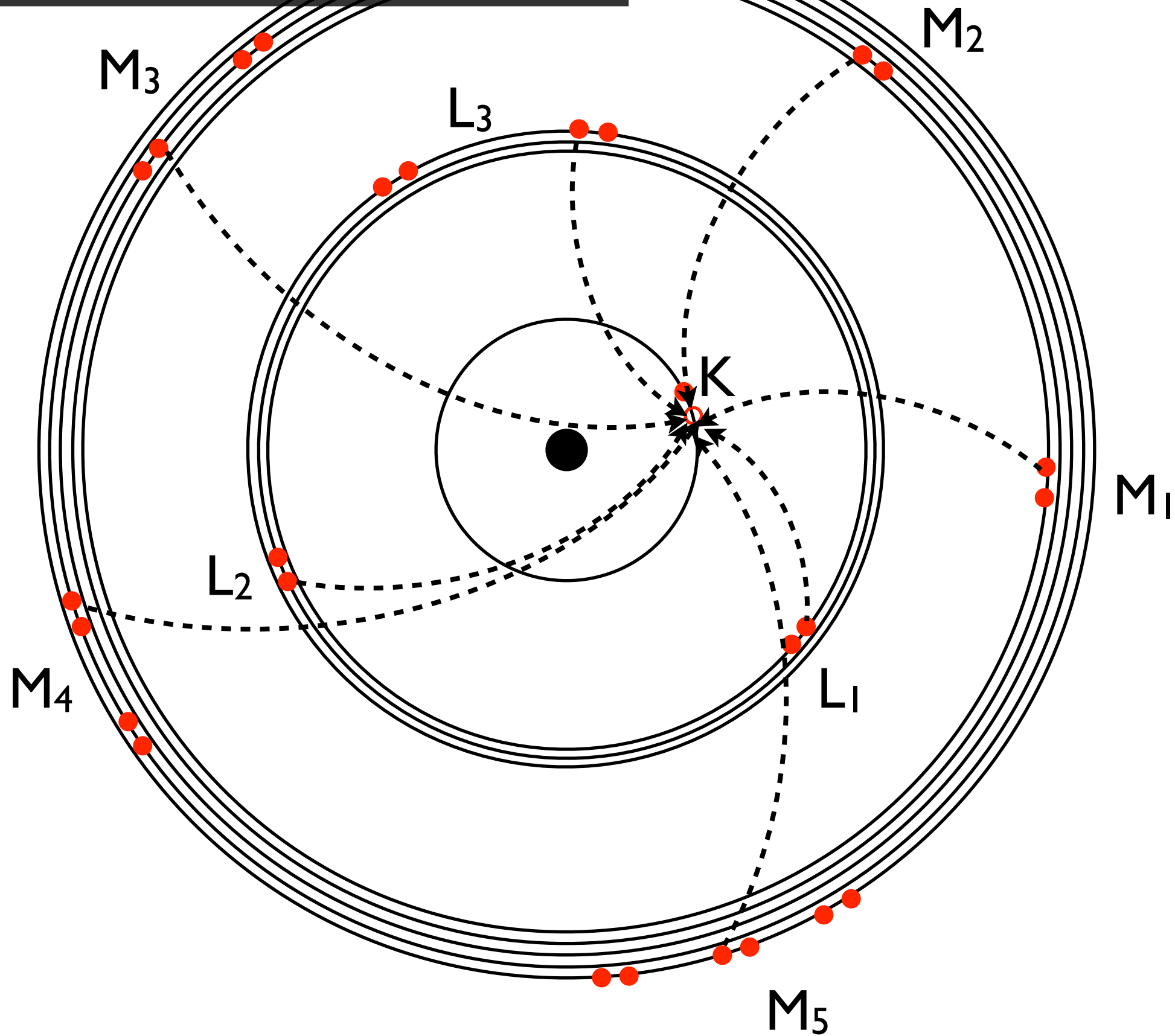


Photoelectric effect: K shell

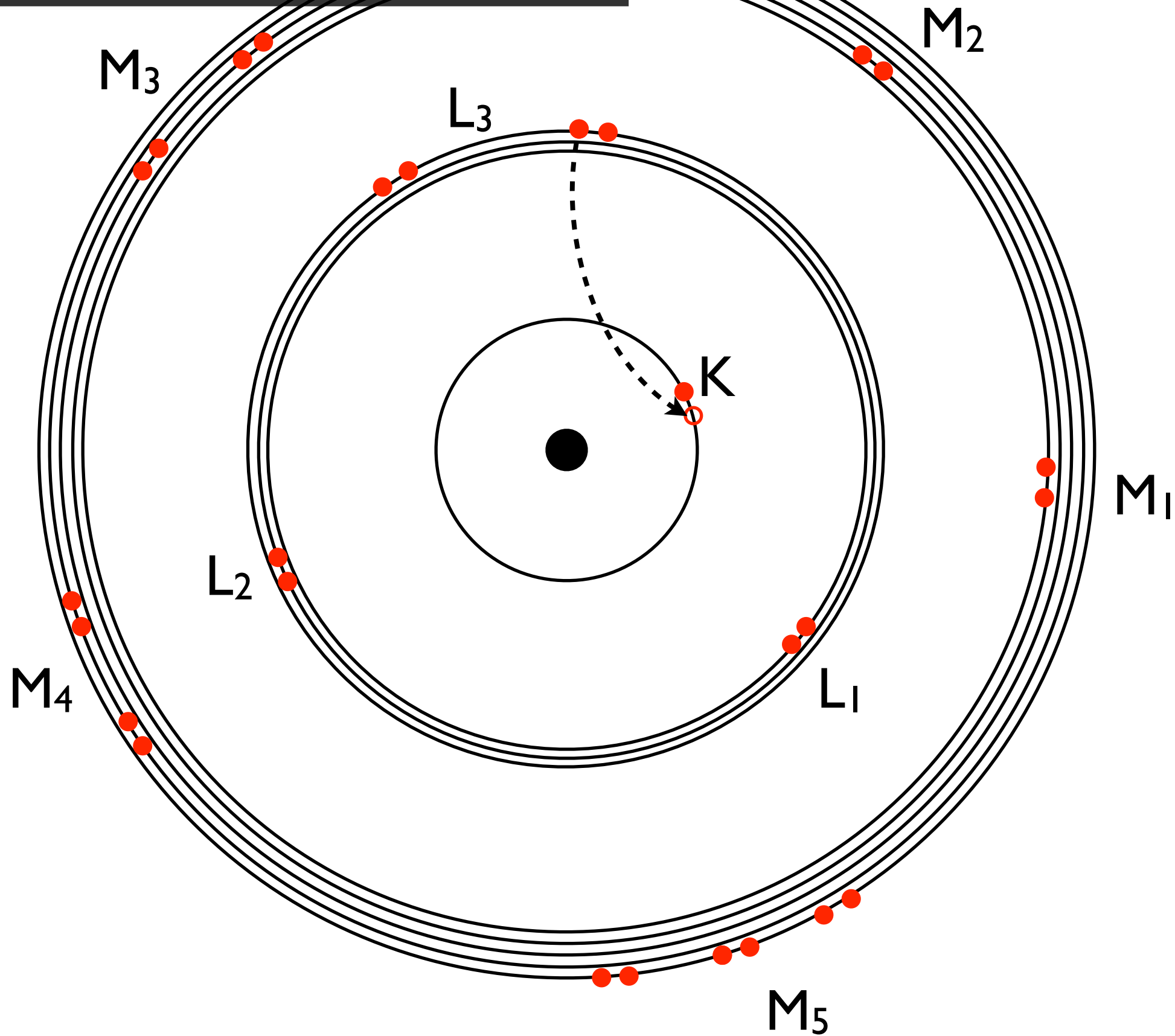


Fluorescence yield: $0 < \omega_K(Z) < 1$
Photon emission or Auger effect?

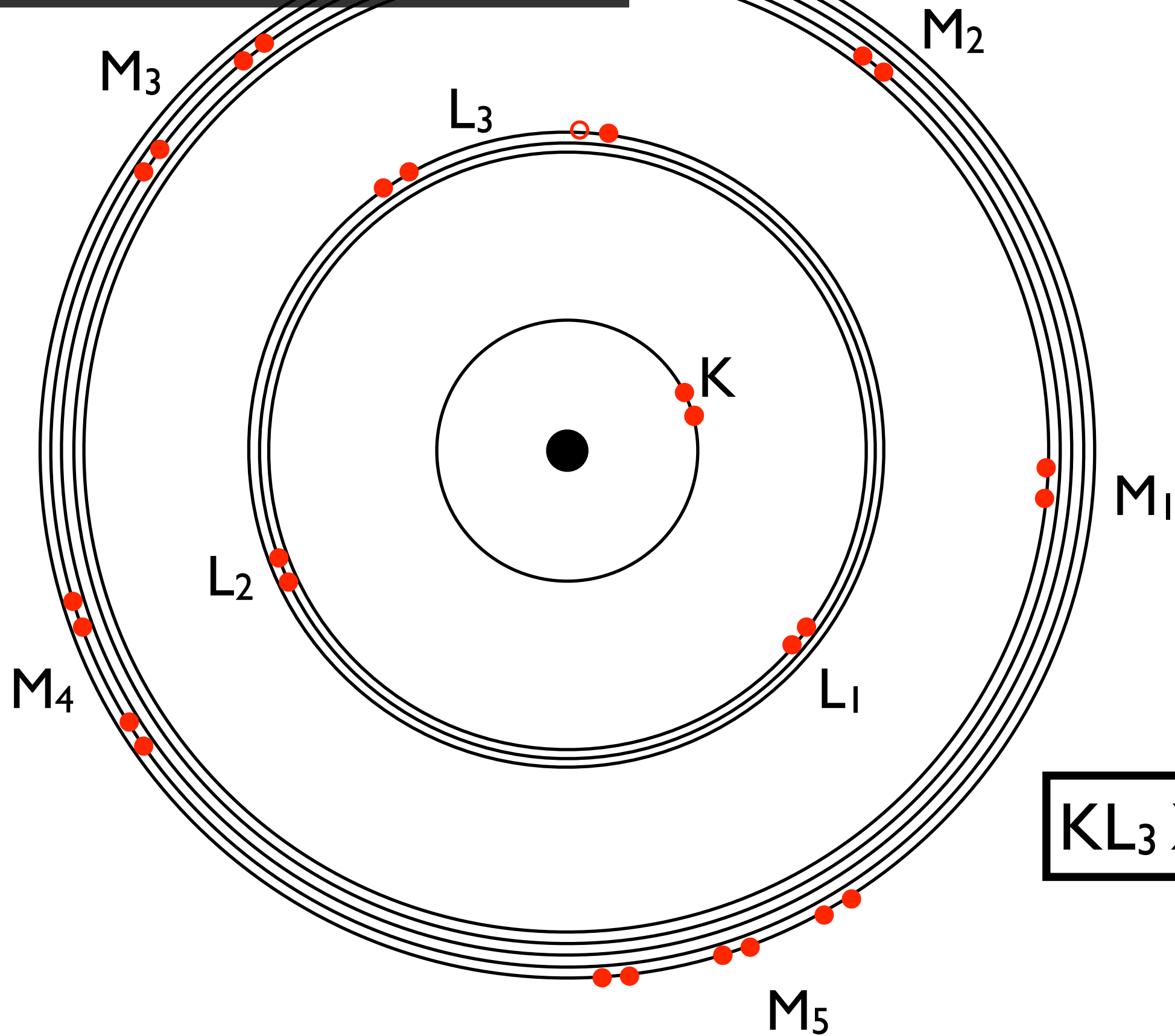
Photoelectric effect: K shell



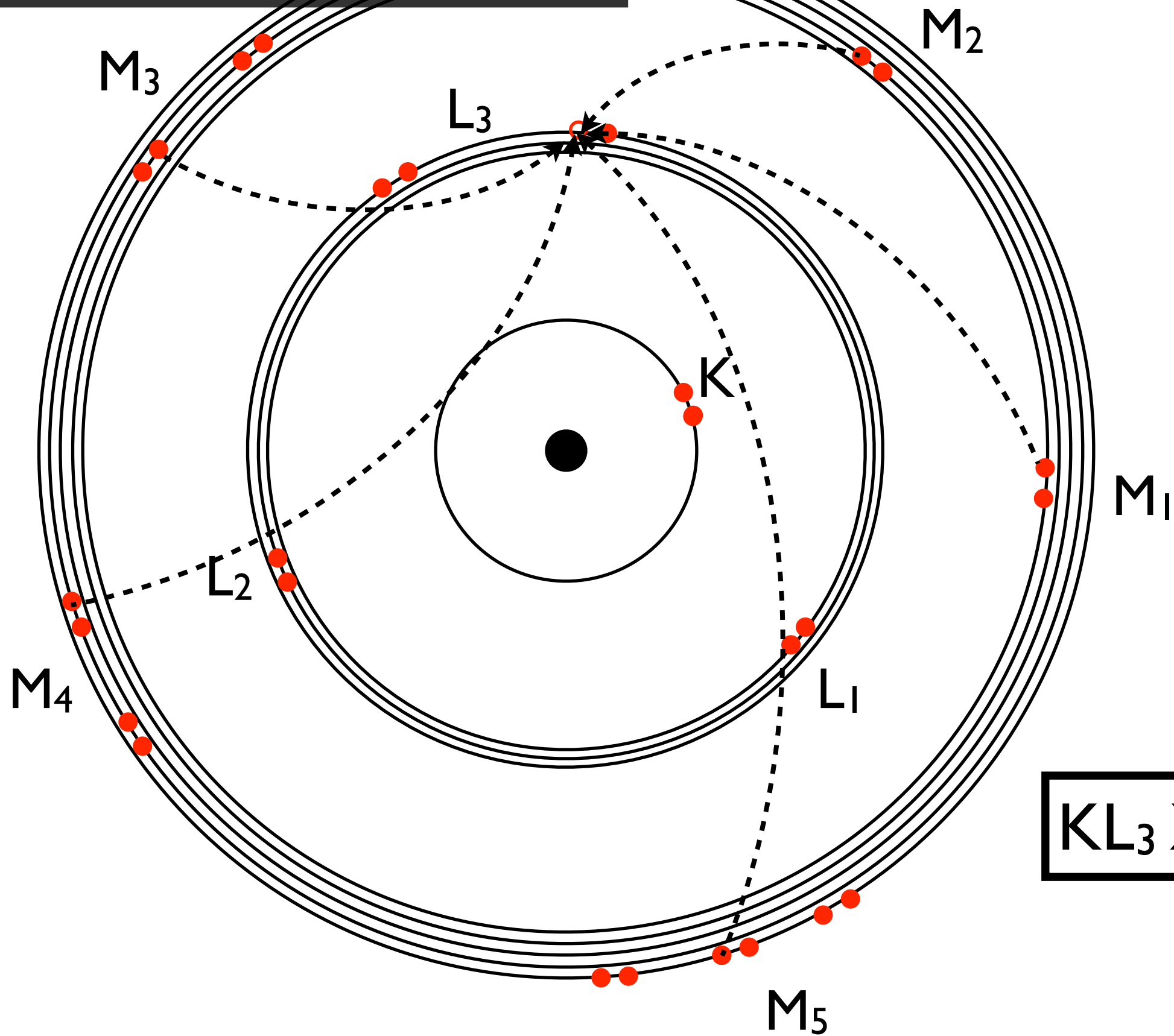
Photoelectric effect: K shell



Photoelectric effect: K shell

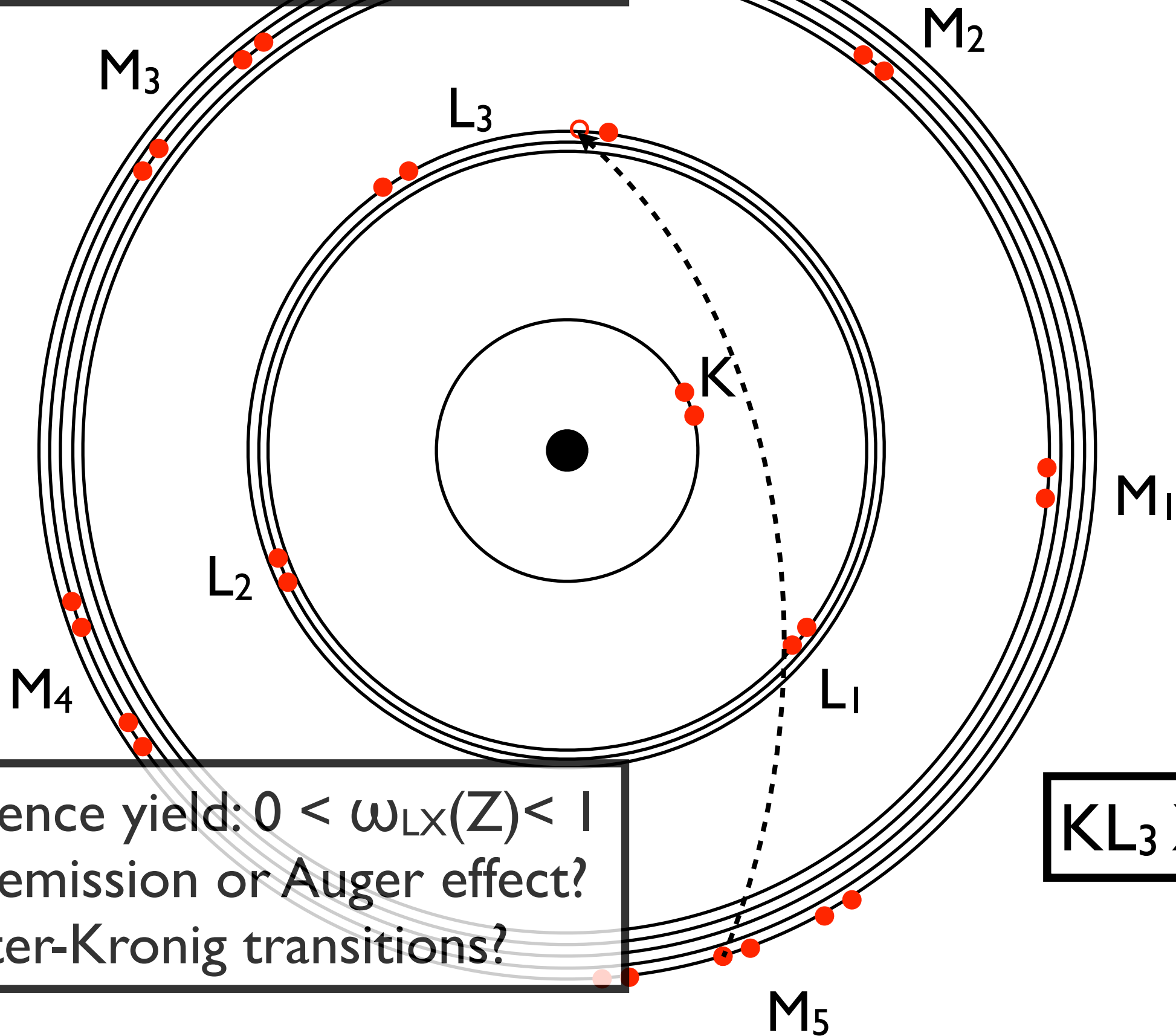


Photoelectric effect: K shell



KL₃ XRF?

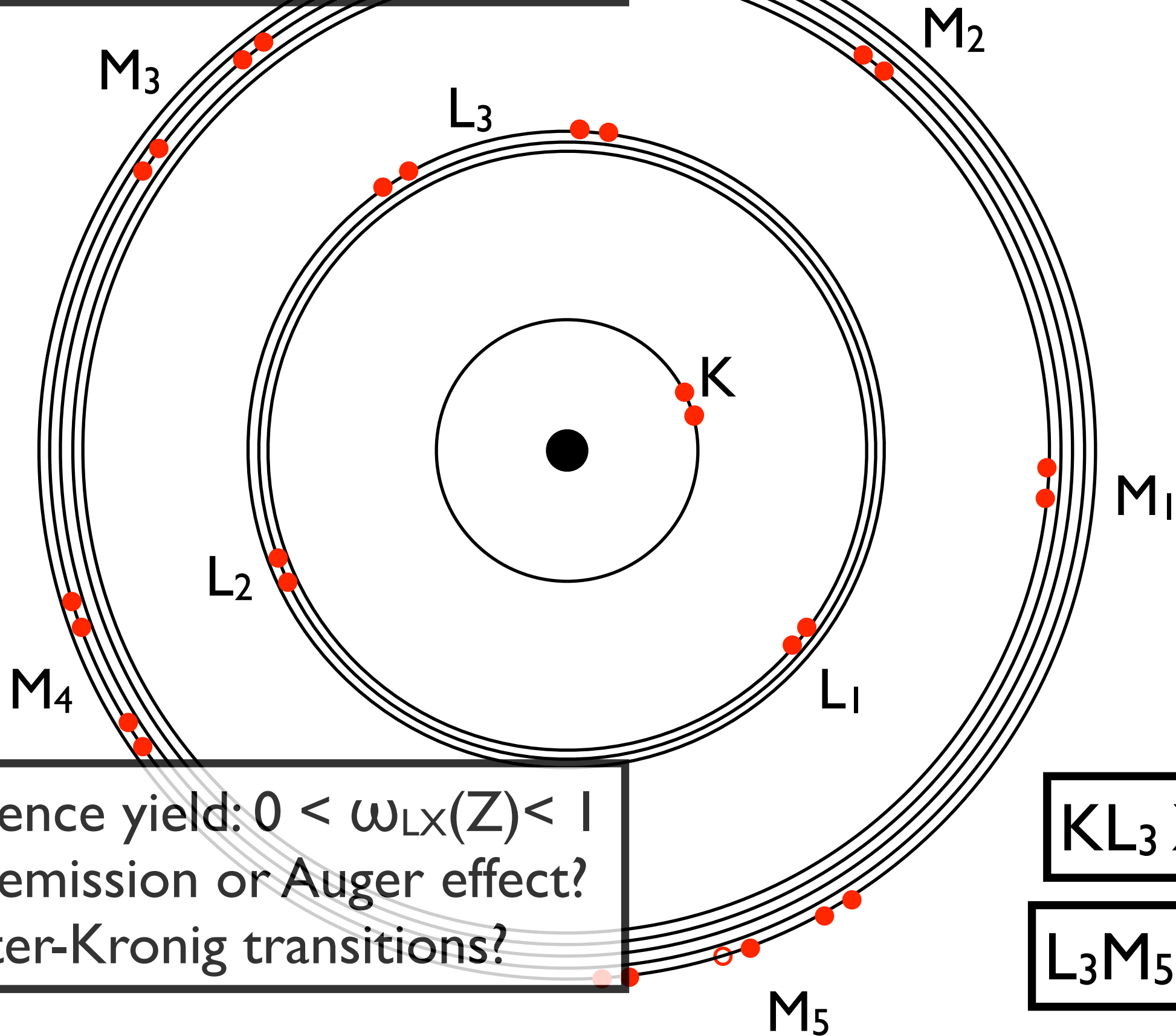
Photoelectric effect: K shell



Fluorescence yield: $0 < \omega_{LX}(Z) < 1$
Photon emission or Auger effect?
Coster-Kronig transitions?

KL₃ XRF?

Photoelectric effect: K shell



Fluorescence yield: $0 < \omega_{LX}(Z) < 1$
Photon emission or Auger effect?
Coster-Kronig transitions?

KL₃ XRF?

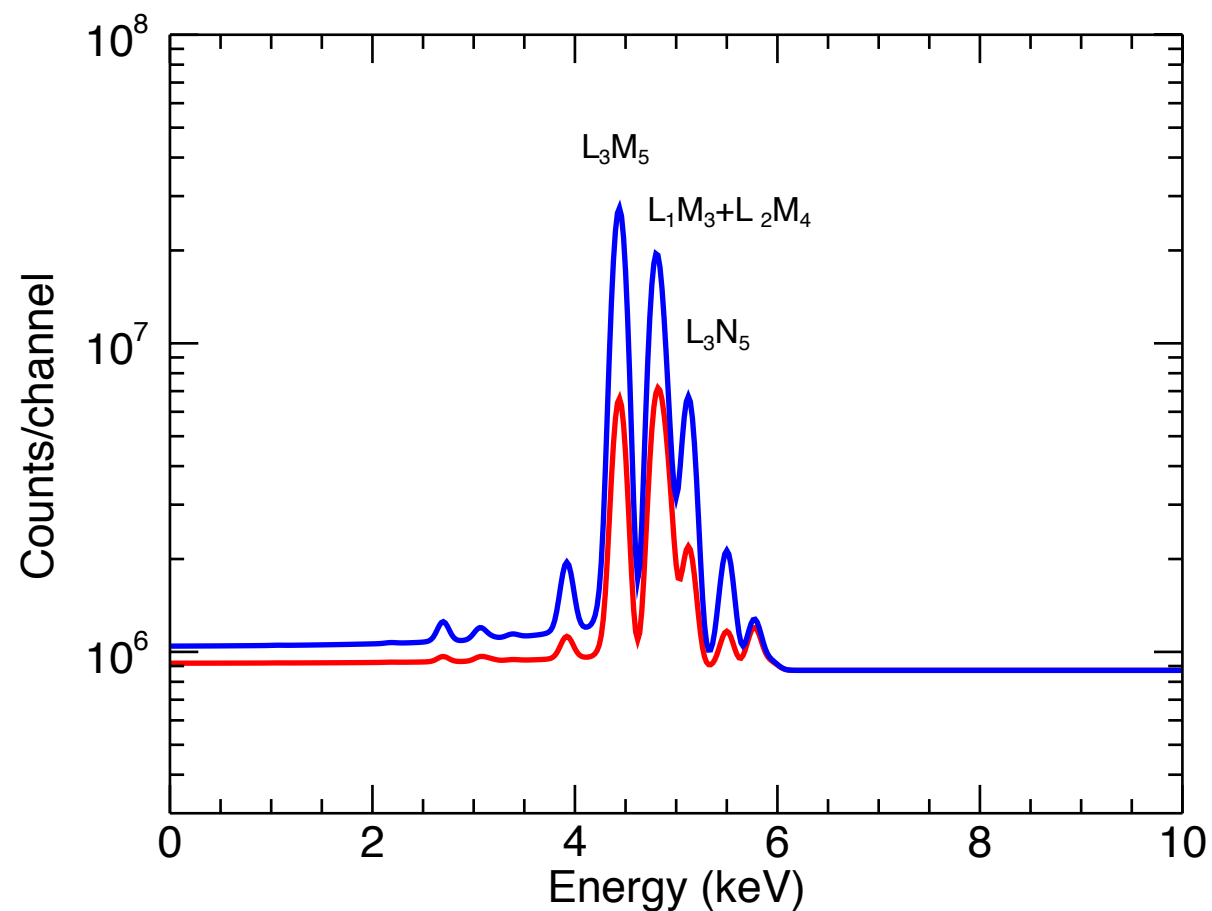
L₃M₅ XRF?

XRF cross sections: cascade effect

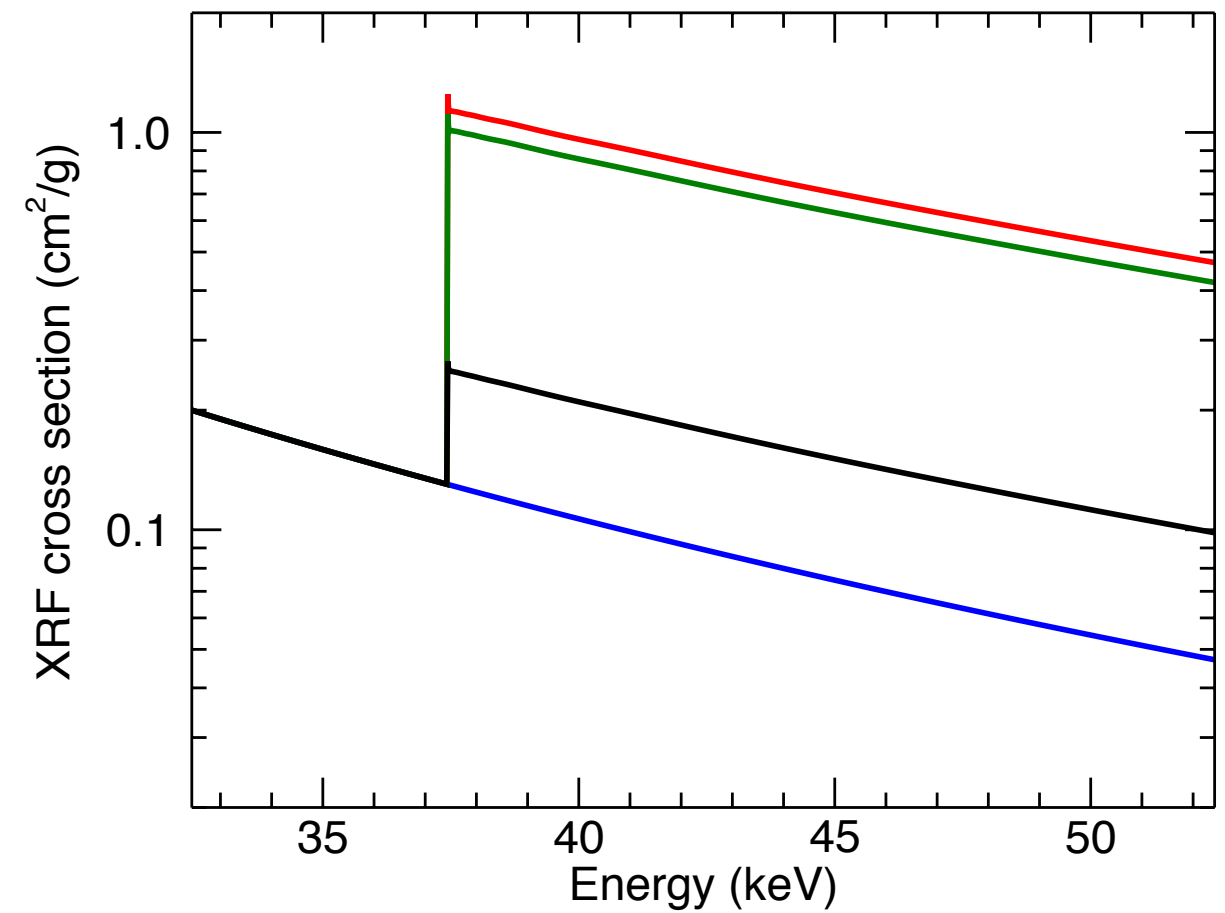
- CS_FluorLine_Kissel_Cascade
- CS_FluorLine_Kissel_Radiative_Cascade
- CS_FluorLine_Kissel_Nonradiative_Cascade
- CSb_FluorLine_Kissel_no_Cascade
- CS_FluorLine_Kissel =
CS_FluorLine_Kissel_Cascade

Allows for the investigation of the separate contributions!

L-lines 40 keV beam

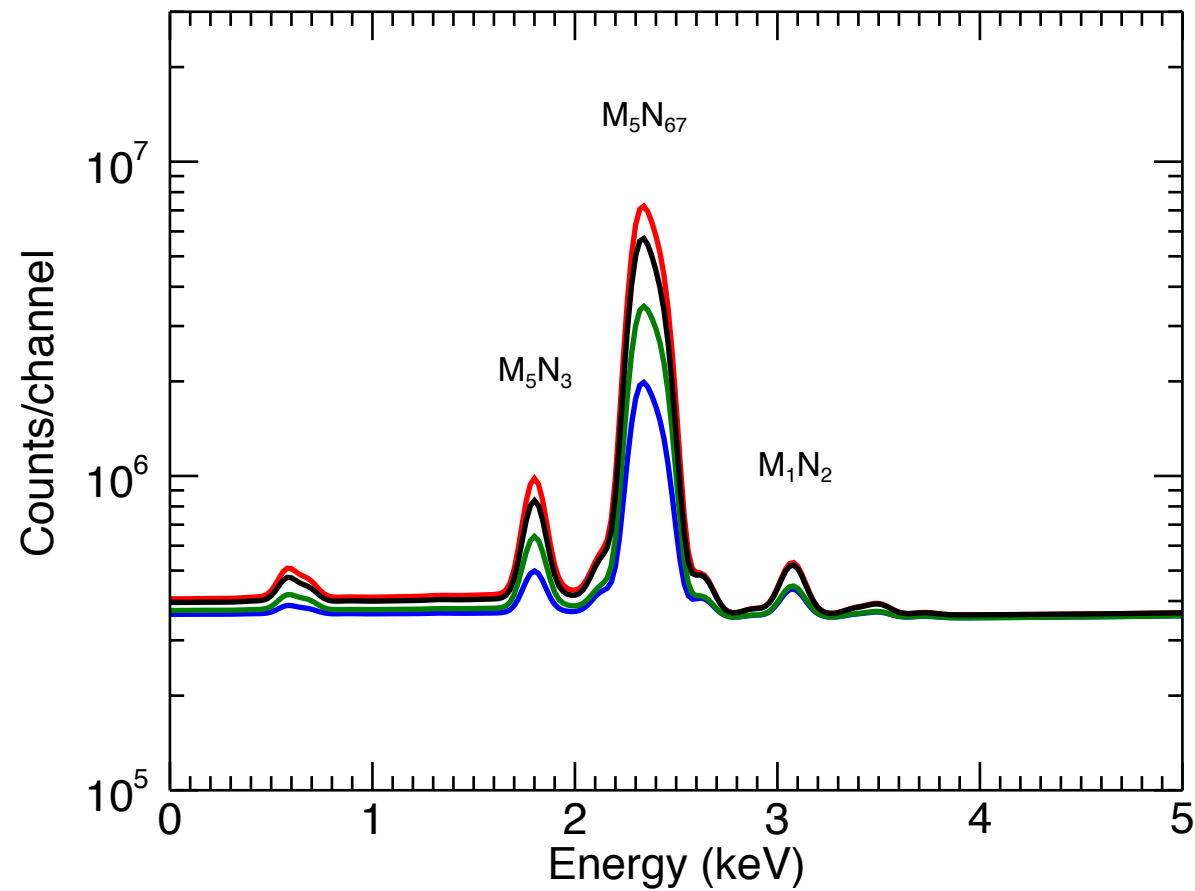


L_3M_5 cross sections around K-edge

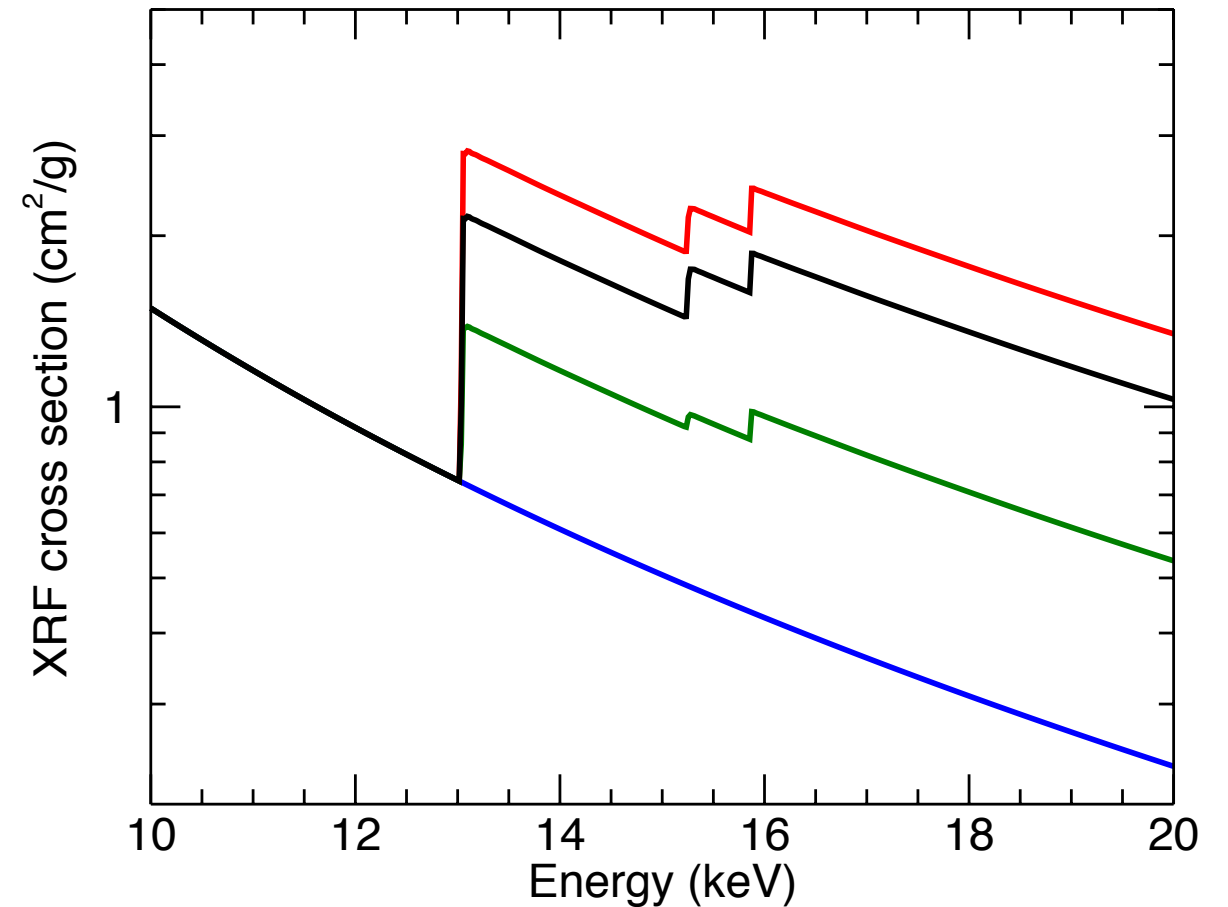


XRF cross sections:
cascade effect: Barium

M-lines 18 keV beam

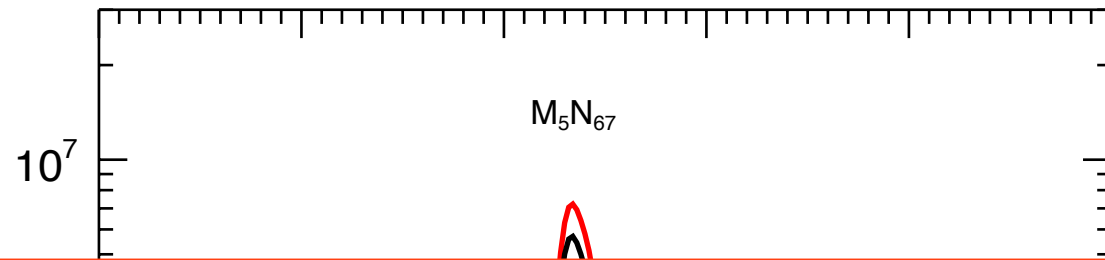


M_5N_7 cross sections around L-edges

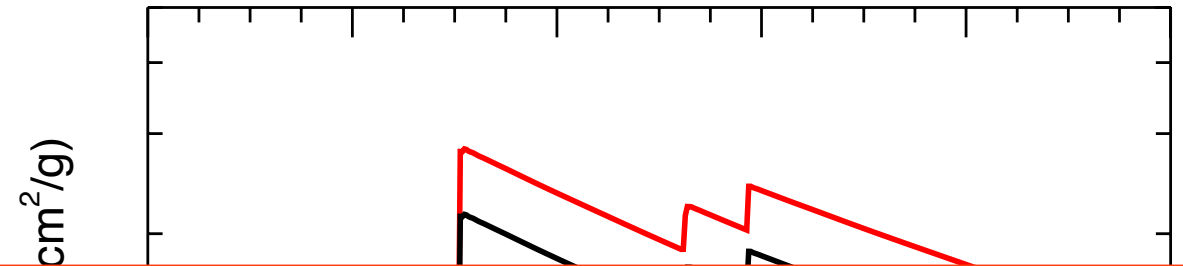


XRF cross sections:
cascade effect: Lead

M-lines
18 keV beam



M_5N_7 cross sections
around L-edges



*Equations of our cascade effect
implementation can be found in our work:*

Schoonjans et al., *Spectrochimica Acta B*, 2011 (66):776-784

XRF cross sections:
cascade effect: Lead

xraylib: platforms

Core library requires only C standard library

- Source tarball
- Linux
 - DEB packages for Debian/Ubuntu
 - RPM packages for RHEL/Centos/Fedora
- Mac OS X
 - MacPorts and Fink
- Windows
 - Windows 32- and 64-bit SDKs available with DLLs
 - MinGW and MinGW-w64: compile from source
 - Cygwin (not supported)
- FreeBSD and Solaris: compile from source (not supported)

xraylib: language bindings

- Core C library
- C++ and Objective-C
- Fortran 2003
- Perl
- Python (2.6+)
- IDL
- Lua
- Java (incomplete)
- Ruby
- PHP
- .NET/C#
- Matlab (unsupported)
- Mathematica (unsupported)
- Labview (unsupported)

xraylib: language bindings vs. platforms

	C C++	Fortran 2003	Python	IDL	Perl	Lua	Ruby	Java	.NET C#	PHP
Linux										
Mac Os X										
Windows (MinGW)										
Windows (32-bit SDK)										
Windows (64-bit SDK)										

xraylib: compilation from source

From UNIX/Linux/MSYS shell:

1. `tar xvfz xraylib-x.y.z.tar.gz`
2. `cd xraylib-x.y.z`
3. `./configure --prefix=... --disable-binding`
4. `make`
5. `make check`
6. `sudo make install`

xraylib: modifying the databases

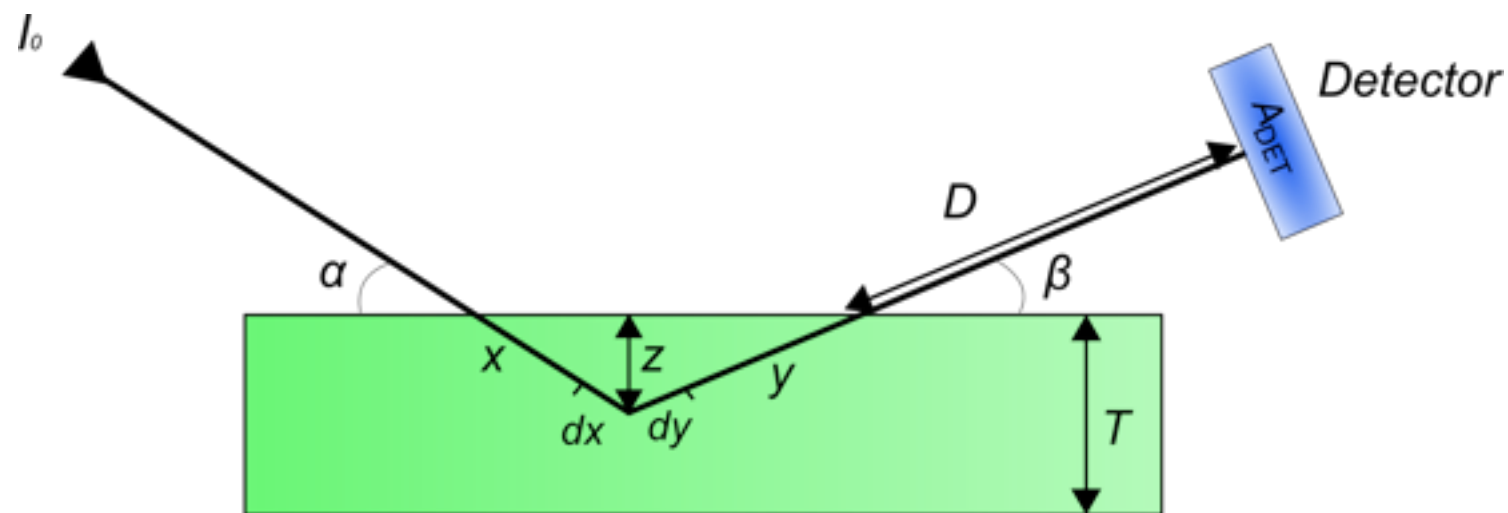
- Databases are turned into code during compilation using bootstrapping
- Changes to the databases need to occur *before* compilation
- Most data files can be easily manipulated
- Cross section files generated through IDL scripts

Typical usage examples

Fundamental parameter method

$$I_{i,K\alpha} = I_0 G w_i Q_{i,K\alpha} \rho T \left(\frac{1 - \exp(-\chi \rho T)}{\chi \rho T} \right)$$

$$\chi = \frac{\mu_0}{\sin \alpha} + \frac{\mu_1}{\sin \beta}$$



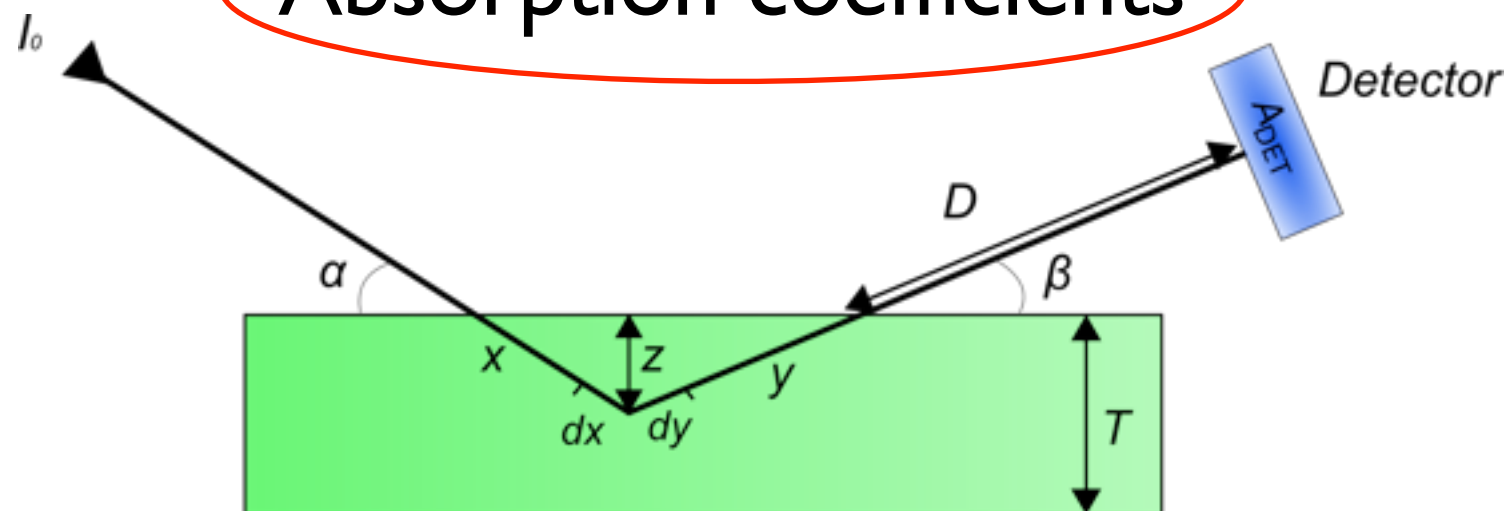
Fundamental parameter method

XRF cross section

$$I_{i,K\alpha} = I_0 G w_i Q_{i,K\alpha} \rho T \left(\frac{1 - \exp(-\chi \rho T)}{\chi \rho T} \right)$$

$$\chi = \frac{\mu_0}{\sin \alpha} + \frac{\mu_1}{\sin \beta}$$

Absorption coefficients



Fundamental parameter method

Parse chemical formula

```
struct compoundData *CompoundParser(  
    const char compoundString[])
```

XRF cross section $Q_{i,K\alpha}$

```
double CS_FluorLine_Kissel(int Z, int line, double energy)
```

Absorption coefficients μ_0 μ_1

```
double CS_Total_CP(char compound[], double energy)
```

```
double LineEnergy(int Z, int line)
```

```

PROGRAM fpm

USE :: xraylib
USE, INTRINSIC :: ISO_C_BINDING

REAL (C_DOUBLE) :: flux = 1E9 !photons/s
REAL (C_DOUBLE) :: G = 1E-5
REAL (C_DOUBLE) :: density = 3.19 !g/cm3
REAL (C_DOUBLE) :: thickness = 0.1 !cm
REAL (C_DOUBLE) :: xrf_intensity, chi
REAL (C_DOUBLE) :: mu_0, mu_1, w_Ca, A_corr, Q
REAL (C_DOUBLE) :: alpha = 45.0, beta = 45.0 !degrees
REAL (C_DOUBLE) :: beam_energy = 20.0 !keV
TYPE (compoundData), POINTER :: cd
CHARACTER (len=50) :: apatite = 'Ca5(P04)3(OH)0.33F0.33Cl0.33'
REAL (C_DOUBLE), PARAMETER :: deg2rad = 3.14159265359/180.0

cd => compoundParser(apatite)
w_Ca = cd%massFractions(6)

mu_0 = CS_Total_CP(apatite, beam_energy)
mu_1 = CS_Total_CP(apatite, LineEnergy(20, KL3_LINE))
chi = mu_0/SIN(deg2rad*alpha) + mu_1/SIN(deg2rad*beta)
A_corr = (1.0-EXP(-chi*density*thickness))/chi*density*thickness
Q = CS_FluorLine_Kissel(20, KL3_LINE, beam_energy)

xrf_intensity = flux*G*Q*w_Ca*density*thickness*A_corr

CALL FreeCompoundData(cd)

WRITE (*, '(A, ES12.4)') 'xrf_intensity: ', xrf_intensity
END PROGRAM fpm

```

Monte-Carlo simulation of an X-ray absorption experiment

Lambert-Beer probability density function

$$f(x) = \mu\rho \exp(-\mu\rho x)$$

Lambert-Beer cumulative distribution function

$$F(x) = \int_0^x \mu\rho \exp(-\mu\rho t) dt = 1 - \exp(-\mu\rho x) \equiv R \Rightarrow x = -\frac{1}{\mu\rho} \ln(1 - R) \Leftrightarrow x = -\frac{1}{\mu\rho} \ln(R)$$

1. With a random number generator, calculate n absorption depths x .
2. Whenever x greater than thickness, add 1 to *transmitted*

Monte-Carlo simulation of an X-ray absorption experiment

Density of a NIST catalog compound

```
struct compoundDataNIST *GetCompoundDataNISTByName(  
    const char compoundString[])
```

Absorption coefficients μ

```
double CS_Total_CP(char compound[], double energy)
```

```

require 'xraylib'

compound = "Uranium Monocarbide"

cdn = Xraylib.GetCompoundDataNISTByName(compound)
density = cdn['density'] #g/cm3
thickness = 0.01 #cm
energy = 50.0 #keV

mu_rho = Xraylib.CS_Total_CP(compound, energy)*density

transmitted = 0
total = 100000

total.times {|i|
  x = -Math.log(rand())/mu_rho
  transmitted += 1 if x > thickness
}

printf("transmitted: %i\n", transmitted)
printf("MC fraction: %f\n", Float(transmitted)/total)
printf("True fraction: %f\n", Math.exp(-mu_rho*thickness))

```

xraylib:
command-line interface

xraylib: command-line interface

- pocket calculator
- python script
- provides access to all functions
- contains documentation
- useful for integration into bash scripts?
- Windows: separate installer available with executable (32-bit only)

xraylib: command-line interface

- `xraylib -h`
list of all available functions
- `xraylib -d`
documentation including database origins
- `xraylib -f $\textit{function\_name}$`
function prototype and argument information

xraylib: command-line interface

- `xraylib "LineEnergy(26,KA1_LINE)"`
- `xraylib "CS_Total_CP('SiO2', 20.0)"`
- `xraylib "GetCompoundDataNISTList()"`
- `xraylib
"Bragg_angle(Crystal_GetCrystal(
'NaCl'), 10.0, 1, 1, 1)"`
- `xraylib "CompoundParser('Na2SO4')"`

xraylib:
web interface

xraylib: web interface

- Website built on top of *xraylib*'s PHP bindings
- Provides syntax examples for all supported bindings
- Incomplete: under development
- Hosted at <http://lvserver.ugent.be/xraylib-web>
- Development at <http://github.com/tschoonj/xraylib-web>

Future developments

Future developments: error handling

- Current implementation is outdated and useless
- Check return values: 0 indicates error or unavailable
- Not thread-safe!
- Consider calling `SetErrorMessage(0)` at the start of your program

Future developments: CUDA and OpenCL

- Running *xraylib* on the GPU
- Huge potential computational gains for Monte-Carlo simulations
- Goal is to produce equivalent device auxiliary functions

Join the development!

<http://github.com/tschoonj/xraylib>

xraylib publications

A. Brunetti, M. Sanchez del Rio, B. Golosio,
A. Simionovici and A. Somogyi,
Spectrochim. Acta Part B, 59, 1725-1731, 2004

T. Schoonjans, A. Brunetti, B. Golosio,
M. Sanchez del Rio, V.A. Solé, C. Ferrero, and L. Vincze,
Spectrochim. Acta Part B, 66, 776–784, 2011.

Acknowledgements

- Laszlo Vincze
- Manuel Sanchez del Rio
- Bruno Golosio
- Antonio Brunetti
- Alexandre Simionovici
- Armando Solé
- Claudio Ferrero
- David Sagan
- Matthew Wormington
- Teemu Ikonen
- Everyone that ever contributed code or reported bugs!

Acknowledgements

- Laszlo Vincze
- Manuel Sanchez del Rio
- Bruno Gc
- Antonio Brunetti
- Alexandre Simionovici
- Armando Solé
- Claudio Ferrero
- David Sagan
- Teemu Ikonen
- Everyone that ever contributed code or reported bugs!

Thank you! Vormington